

# Connecting the publishing platforms

Everything a brand can publish to, what each one needs from you, and where it stands today.

All fourteen platforms publish through native adapters. The Ayrshare aggregator is still there as a switchable fallback — see [The aggregator switch](#) — but nothing depends on it.

## Where things stand

State as of the current `apps/api/.env`. "Configured" means this deployment holds the app credentials; whether a *brand* has connected an account is a separate question, answered per brand in **Settings** → **Account Connection**.

### Working

| PLATFORM              | NOTES                                                                                |
|-----------------------|--------------------------------------------------------------------------------------|
| <b>X</b>              | Connected and publishing. Access tokens last 2 hours and now refresh automatically.  |
| <b>YouTube Shorts</b> | Connected and publishing. Google's tokens last 1 hour and now refresh automatically. |

### Configured, but no account connected yet

| PLATFORM      | WHAT IS LEFT                                                                                                                                                                      |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TikTok</b> | Keys are set. The connect flow reaches TikTok's consent screen and fails there on <code>client_key</code> — that error means the redirect URI, see <a href="#">Redirect URI</a> . |

### Come free — no setup, ever

These four have no developer app and no Connect button, because they are not separate accounts. They publish through a connection another platform already made.

| PLATFORM                   | PUBLISHES THROUGH | WHY                                                                                                    |
|----------------------------|-------------------|--------------------------------------------------------------------------------------------------------|
| <b>Instagram Stories</b>   | Instagram         | Same account, same token; a story is <code>media_type=STORIES</code> on the container.                 |
| <b>Facebook</b>            | Instagram         | An Instagram Business account exists because a Facebook Page owns it. Meta issues both from one login. |
| <b>Facebook Groups</b>     | Instagram         | A Group that same Page identity administers. Needs a group id per post.                                |
| <b>YouTube (long-form)</b> | YouTube Shorts    | Same channel, same token. "Shorts" is a duration and an aspect ratio, not a destination.               |

Connect Instagram and three of these light up at once.

## Needs a new scope — reconnect required

| PLATFORM         | SCOPE<br>ADDED                  | WHO IS AFFECTED                                                                                                                                                                                                                                                                                                                                       |
|------------------|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Instagram</b> | <code>pages_manage_posts</code> | Any brand that connected Instagram <b>before</b> Facebook publishing shipped holds a token without this scope. Publishing to Instagram still works; publishing to the Facebook Page fails with a Graph permissions error. The fix is reconnecting in Settings — the adapter's error message says so rather than leaving you to decode error code 200. |

No other platform's scopes changed. X already requested `offline.access`, which is what makes its refresh work.

## Not configured yet

| PLATFORM                                       | KEYS NEEDED                                                                            | APPROVAL GATE                                                                                 |
|------------------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Instagram</b> (+ Facebook, Stories, Groups) | <code>META_APP_ID</code> ,<br><code>META_APP_SECRET</code>                             | Yes — App Review for content publishing                                                       |
| <b>LinkedIn</b>                                | <code>LINKEDIN_CLIENT_ID</code> ,<br><code>LINKEDIN_CLIENT_SECRET</code>               | Yes — Community Management API access                                                         |
| <b>Threads</b>                                 | <code>THREADS_APP_ID</code> ,<br><code>THREADS_APP_SECRET</code>                       | Yes — App Review                                                                              |
| <b>Pinterest</b>                               | <code>PINTEREST_APP_ID</code> ,<br><code>PINTEREST_APP_SECRET</code>                   | Trial access works immediately on your own account; Standard needs review                     |
| <b>Reddit</b>                                  | <code>REDDIT_CLIENT_ID</code> ,<br><code>REDDIT_CLIENT_SECRET</code>                   | No                                                                                            |
| <b>Google Business</b>                         | <code>GOOGLE_BUSINESS_CLIENT_ID</code> ,<br><code>GOOGLE_BUSINESS_CLIENT_SECRET</code> | <b>Yes, and it is absolute</b> — every API call returns 403 until Google approves the project |

## Needs nothing from you

| PLATFORM       | HOW IT CONNECTS                                                                                                                                                      |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Bluesky</b> | No developer app exists. Each brand connects with their own handle and an <b>app password</b> , created in their Bluesky settings. The adapter is always registered. |

## The redirect URI: one value every console must match

There is **one** `SOCIAL_REDIRECT_URI` shared by every OAuth platform, so the same string has to be registered, character for character, in each console.

Current local value:

```
http://127.0.0.1:8080/oauth/social/callback
```

On staging it is set by the deploy to `https://<api-host>/oauth/social/callback`.

**127.0.0.1 and localhost are different strings to every provider.** A mismatch is rejected on the provider's own consent screen before any code comes back, and only one of them tells you that:

| PROVIDER | WHAT IT SAYS                        | WHAT IT MEANS             |
|----------|-------------------------------------|---------------------------|
| Google   | Error 400: redirect_uri_mismatch    | the URI is not registered |
| TikTok   | "correct the following: client_key" | the URI is not registered |
| X        | "You weren't able to give access"   | the URI is not registered |

Treat TikTok's `client_key` complaint as a redirect-URI problem until proven otherwise. If a console refuses to save a plain-`http` loopback URI, that provider needs HTTPS — point `SOCIAL_REDIRECT_URI` at an HTTPS tunnel to port 8080 and register the tunnel's `/oauth/social/callback` everywhere instead.

One more shared value: `OAuthStateSecret`. Every connect flow is gated on it — without it `integration.connect` is not registered at all and no brand can connect anything. Generate one with `openssl rand -base64 32`. It is already set locally and on staging.

## Per-platform setup

### Meta — Instagram, Instagram Stories, Facebook, Facebook Groups

One app serves all four.

1. **developers.facebook.com** → My Apps → Create App → type **Business**.
2. Add the **Facebook Login** product. Under its settings, add `SOCIAL_REDIRECT_URI` to *Valid OAuth Redirect URIs*.
3. Add the **Instagram Graph API** product.
4. Request these permissions under App Review:
  - `instagram_content_publish` — posting to Instagram
  - `pages_show_list`, `pages_read_engagement` — finding the Page
  - `pages_manage_posts` — posting to the Facebook Page
5. Copy **App ID** and **App Secret** from Basic settings into `META_APP_ID` and `META_APP_SECRET`.
6. The Instagram account must be a **Business or Creator** account and must be linked to a Facebook Page. A personal Instagram account cannot publish through any API.

**Facebook Groups** additionally needs Meta's Groups API permission *and* the Group must have installed your app. Meta restricted this heavily in 2020, so expect it to refuse for most brands. The adapter says so rather than failing vaguely.

## TikTok

1. **developers.tiktok.com** → Manage apps → your app.
2. Add the **Login Kit** product, and under it register `SOCIAL_REDIRECT_URI` as a Redirect URI.
3. Add the **Content Posting API** product.
4. Scopes: `video.publish` , `user.info.basic` .
5. Client Key → `TIKTOK_CLIENT_KEY` , Client Secret → `TIKTOK_CLIENT_SECRET` .

Already configured here. The current failure is at step 2 — see [Redirect URI](#).

## LinkedIn

1. **linkedin.com/developers** → Create app, associated with a Company Page.
2. Under **Auth**, add `SOCIAL_REDIRECT_URI` to Authorized redirect URLs.
3. Under **Products**, request **Share on LinkedIn** and **Sign In with LinkedIn using OpenID Connect**. Scopes: `w_member_social` , `openid` , `profile` .
4. Client ID → `LINKEDIN_CLIENT_ID` , Client Secret → `LINKEDIN_CLIENT_SECRET` .

LinkedIn tokens last 60 days and **cannot be refreshed** unless you are in their partner programme, so brands reconnect every two months. That is LinkedIn's constraint, not a gap here.

## X

1. **developer.x.com** → Projects & Apps → your app.
2. **User authentication settings** → set up:
  - App permissions: **Read and write**
  - Type of App: **Web App, Automated App or Bot** (this is the confidential client type; the refresh flow depends on it)
  - Callback URI: `SOCIAL_REDIRECT_URI`
3. Use the **OAuth 2.0** Client ID and Client Secret — *not* the API Key/Secret from the 1.1 section. They live further down the Keys and tokens page.
4. → `X_API_KEY` , `X_API_SECRET` .

Scopes requested include `offline.access` , which is what makes the 2-hour token refreshable. Without it X returns no refresh token.

## YouTube — Shorts and long-form

1. **console.cloud.google.com** → APIs & Services → **Enable** the *YouTube Data API v3*.
2. Credentials → Create credentials → **OAuth client ID** → Web application.
3. Add `SOCIAL_REDIRECT_URI` under **Authorized redirect URIs**.
4. OAuth consent screen: add scopes `youtube.upload` and `youtube.readonly`. While the app is in Testing, add each connecting Google account as a Test user.
5. → `YOUTUBE_CLIENT_ID`, `YOUTUBE_CLIENT_SECRET`.

## Threads

**Its own app — not the Meta one above.** Threads has a separate OAuth host ( `threads.net` ), a separate graph, and separate scopes. A Meta app token does not authenticate against it.

1. **developers.facebook.com** → Create App → use case **Threads API**.
2. Threads API → Settings → add `SOCIAL_REDIRECT_URI` as a *Redirect Callback URL*.
3. Request `threads_basic` and `threads_content_publish`. Both need App Review before anyone outside your own test users can connect.
4. App ID / App Secret → `THREADS_APP_ID`, `THREADS_APP_SECRET`.

## Pinterest

1. **developers.pinterest.com** → My apps → Create app.
2. Add `SOCIAL_REDIRECT_URI` under the app's Redirect URIs.
3. Scopes: `pins:write` and `boards:read`. `boards:read` is not optional — a pin must name a board, and that is how the list is offered.
4. → `PINTEREST_APP_ID`, `PINTEREST_APP_SECRET`.

A new app has **Trial access**, which only works against your own Pinterest account. Standard access needs a review submission.

Every pin needs a board id set on the campaign's accounts. There is no default, and the adapter refuses rather than guessing.

## Reddit

1. **reddit.com/prefs/apps** → *create another app* → type **web app**.
2. **redirect uri** must equal `SOCIAL_REDIRECT_URI` exactly.
3. **Tick the reCAPTCHA**. The form will not submit without it, and Reddit does not say so — the button simply does nothing.
4. The **client id is the unlabelled string directly under the app name**; the secret is the field marked *secret*. Mixing these up is the usual first failure.
5. → `REDDIT_CLIENT_ID`, `REDDIT_CLIENT_SECRET`.

If **"create app" refuses**, the line about the *Responsible Builder Policy* is Reddit's standing footer, not an error — it is on the page before you touch anything. Two things actually block the button:

1. the unticked reCAPTCHA, which is the usual one; and
2. the *"you must also **register to use the API**"* link in the same sentence. Since 2023 Reddit requires a separate API registration, and an account that has not completed it cannot create an app. Follow that link, complete it, then come back to this form.

Neither is something this product can do for you, and neither is a code problem — the adapter and OAuth flow are built and waiting for the two keys.

These two keys are also read by the trend sources. One app, two uses, one pair.

Every post needs a subreddit set on the campaign's accounts. Reddit is the platform where posting to the wrong community costs the most, so there is no default and the adapter refuses without one.

## Google Business Profile

**A second Google OAuth client, separate from YouTube's.** The scope is different and access is granted per project, so keeping them apart means a Business review cannot put YouTube publishing at risk.

1. **console.cloud.google.com** → Credentials → Create OAuth client ID → Web application.
2. Add `SOCIAL_REDIRECT_URI` under Authorized redirect URIs.
3. Enable **My Business Account Management API** and **My Business Business Information API**.
4. **Request API access** at `support.google.com/business/contact/api_access`. Until Google approves the project, every call returns 403. This approval is the long pole, not the code.
5. → `GOOGLE_BUSINESS_CLIENT_ID`, `GOOGLE_BUSINESS_CLIENT_SECRET`.

Every post needs a location. A profile with several branches has no default one.

## Bluesky

Nothing to configure. Each brand connects their own account:

1. In Bluesky: **Settings** → **Privacy and Security** → **App Passwords** → **Add App Password**. Copy the `xxxx-xxxx-xxxx` value.
2. In SparkSocial, connect Bluesky with the handle ( `name.bsky.social` ) and that app password.

Use an **app password**, never the account password. App passwords are scoped and revocable on their own. The credential is verified against Bluesky before it is stored, so a typo fails at the form rather than at the first scheduled post.

Bluesky posts are text-only for now — attaching images needs a blob upload, which is not built. The post goes out without the image rather than silently dropping it.

---

## The aggregator switch

Ayrshare covers all fourteen platforms with one API key and no per-brand OAuth. Two flags:

```
AYRSHARE_API_KEY=...           # the vendor key
PUBLISH_USE_AGGREGATOR=true     # in the routing table, BEHIND the natives
PUBLISH_PREFER_AGGREGATOR=true  # AHEAD of the natives – the real switch
```

Routing picks the first adapter that supports a platform. Now that all fourteen are native,

`PUBLISH_USE_AGGREGATOR` alone changes nothing — nothing ever reaches it.

`PUBLISH_PREFER_AGGREGATOR=true` **is the lever**: it moves every platform onto Ayrshare without touching a connection, a token or a deploy, and removing it moves them all back. Worth pulling when a native app is rate-limited, awaiting re-approval, or misbehaving.

The API logs which order is live at boot.

---

## After connecting: what to expect

**Tokens refresh themselves.** TikTok, X, YouTube, Pinterest, Reddit and Google Business all renew automatically — on a 15-minute clock, and again immediately before any publish that would otherwise use an expiring token.

**Three platforms cannot refresh**, by the provider's design, so brands reconnect by hand:

| PLATFORM             | TOKEN LIFE | WHY NO REFRESH                                                 |
|----------------------|------------|----------------------------------------------------------------|
| Instagram / Facebook | ~60 days   | Meta issues no refresh token                                   |
| Threads              | ~60 days   | Same; renewal is a different call against a different lifetime |
| LinkedIn             | ~60 days   | Refresh tokens are gated behind LinkedIn's partner programme   |

Bluesky never expires — the app password is valid until revoked in Bluesky.

**Settings** → **Account Connection** shows each platform's adapter ( `native:x` vs `aggregator:stub` ) and connection health, and warns before a token dies.

---

## Trend sources

Separate from publishing, and separate keys. Publishing is *where posts go out*; trend sources are *where ideas come in* — what `trend.rank` and the Discovery screen read to suggest what to post about.

**Two are keyless and already live.** With no keys at all the product still discovers trends; the rest widen the pool.

### Where they stand

| SOURCE        | STATE | KEY            |
|---------------|-------|----------------|
| Google Trends | live  | none — keyless |
| Hacker News   | live  | none — keyless |

| SOURCE              | STATE          | KEY                                                    |
|---------------------|----------------|--------------------------------------------------------|
| <b>YouTube</b>      | live           | YOUTUBE_API_KEY                                        |
| <b>Product Hunt</b> | live           | PRODUCTHUNT_CLIENT_ID / _SECRET                        |
| <b>Reddit</b>       | not configured | REDDIT_CLIENT_ID / _SECRET                             |
| <b>X</b>            | not configured | X_BEARER_TOKEN                                         |
| <b>TikTok</b>       | not configured | TIKTOK_CREATIVE_TOKEN — needs a Marketing API approval |
| <b>Pinterest</b>    | not configured | PINTEREST_ACCESS_TOKEN                                 |

An unset source is not an error. It is skipped, and the ones that are set carry the ranking — which is why four being live already means Discovery works today.

## Setting up the four that are not

Numbered the same way as the publishing platforms above, because these are ordinary vendor consoles too — the only difference is that a trend key belongs to the *deployment* rather than to a brand, so there is no Connect button and no consent screen. You set the key, restart the API, and the source joins the pool.

### Reddit (trends)

Nothing separate to do. The trend source reads the **same** REDDIT\_CLIENT\_ID / REDDIT\_CLIENT\_SECRET as Reddit publishing, so completing [Reddit](#) above turns both on at once. One app, one registration, two uses.

### X (trends)

A **bearer token**, which is a different credential from the OAuth client used for publishing — app-level, no user, no consent screen.

1. **developer.x.com** → your Project → your App → **Keys and tokens**.
2. Under *Authentication Tokens*, find **Bearer Token** and **Generate**.
3. Copy it immediately — X shows it once, and regenerating invalidates the old one.
4. → X\_BEARER\_TOKEN .

Optional tuning: X\_TREND\_WOEID sets the "where on earth" region for trends (1 is worldwide, the default).

**X's free tier does not include the search endpoints this source reads.** A free bearer token authenticates and then gets 403s, which look like a broken integration and are not. This is the one source it is most reasonable to leave off unless you already pay for a tier that includes recent search.

### TikTok (trends)

A **Creative Center** token from TikTok's Marketing API — **not** the Login Kit client key used for publishing. It is a separate application with its own approval, and the two are not interchangeable.

1. **business-api.tiktok.com** → register a developer account.

2. Create an app and request **Creative Center / Trends** access.
3. Generate a long-lived access token once approved.
4. → `TIKTOK_CREATIVE_TOKEN` .

Optional tuning: `TIKTOK_TREND_REGION` (an ISO country code) and `TIKTOK_TREND_PERIOD_DAYS` (the lookback window).

Approval here is a real wait and the source is genuinely optional — Google Trends and Hacker News cover general discovery without it.

## Pinterest (trends)

An access token from the **same Pinterest app** as publishing, but a different kind of credential: this one belongs to the deployment and reads public trends, where the publishing connection belongs to a brand and posts on its behalf.

1. **developers.pinterest.com** → your app (the one from [Pinterest](#) above, or a new one).
2. Generate an access token with a read scope.
3. → `PINTEREST_ACCESS_TOKEN` .

## The two that are already live, for reference

**YouTube** — `YOUTUBE_API_KEY` is a plain API key, not the OAuth client used for publishing:  
**console.cloud.google.com** → Credentials → Create credentials → **API key**, with the *YouTube Data API v3* enabled on the project. Already set.

**Product Hunt** — **api.producthunt.com/v2/oauth/applications** → create an application →  
`PRODUCTHUNT_CLIENT_ID` / `PRODUCTHUNT_CLIENT_SECRET` . Already set.

## Checking that a source actually works

A trend key is not like a Connect button: there is no consent screen to succeed or fail, so nothing tells you whether the value you pasted is right. Worse, the composite catches every per-source failure deliberately — one vendor going down degrades the feed to "everyone else" instead of taking it down — so a wrong, expired or under-scoped token produces exactly the same feed as one that was never pasted. The boot banner lists which sources are *configured*, not which ones work.

After setting any key, restart the API and run:

```
npx tsx --env-file-if-exists=apps/api/.env scripts/check-trend-sources.mts
```

It fetches from every configured source and prints one line each:

| RESULT | WHAT IT MEANS                                                         |
|--------|-----------------------------------------------------------------------|
| ok     | Real trends came back, with a sample so you can see they are real     |
| EMPTY  | Reachable, returned nothing — usually a token without the right scope |

| RESULT         | WHAT IT MEANS                                                     |
|----------------|-------------------------------------------------------------------|
| FAILED         | The vendor rejected the call; the message is the vendor's own     |
| not configured | No credential set, and the exact variable names that would set it |
| disabled       | Credential present, switch deliberately off                       |

EMPTY is reported separately from ok on purpose. A wrong token, a missing scope and a genuinely quiet day are indistinguishable from an empty result, and treating them as success is how a dead source stays unnoticed. That matters most for TikTok and Pinterest, which both fail this way rather than erroring.

This is not hypothetical. Google Trends was configured, enabled, listed in the boot banner and returning nothing for an unknown length of time, because GOOGLE\_TREND\_REGION had been given a copy of the TREND\_REGIONS line. The two variables are different — the first is a single fallback geo, the second is the multi-region breakdown, which is asked one region at a time — and the vendor's 400 named neither. The check found it on its first run.

### Turning a source off on purpose

Two sources have an explicit switch, because both can cost money and both are worth disabling independently of whether a key is present:

```
TREND_SOURCE_X_ENABLED=false  
TREND_SOURCE_TIKTOK_ENABLED=false
```

## Choosing which account gets connected

**Only Google asks.** Connecting YouTube or Google Business shows an account chooser every time — SparkSocial sends `prompt=select_account`, so it asks even when one account is signed in.

**Every other platform connects whichever account the browser is already signed in to, and never asks.** X, TikTok, LinkedIn, Meta, Threads, Pinterest and Reddit publish no documented way to request an account chooser on an OAuth 2.0 authorize URL. The browser session decides.

That matters most for agencies: connecting a client's account from your own laptop, while signed in to your own, silently connects yours.

Two things guard against it:

1. **Check what it says.** The confirmation names the account — "X connected as @clienthandle." — not just the platform. If that is the wrong handle, Disconnect and reconnect.
2. **Connect from the right session.** Before connecting, either sign out of that platform in this browser, or use a private/incognito window and sign in as the account you want.

Bluesky has no ambiguity: you type the handle yourself.

## Troubleshooting

| SYMPTOM                                                 | CAUSE                                                                                  |
|---------------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>redirect_uri_mismatch</code> (Google)             | The URI is not registered, or <code>localhost</code> vs <code>127.0.0.1</code>         |
| " <i>correct the following: client_key</i> " (TikTok)   | Same — the redirect URI, despite what it says                                          |
| " <i>You weren't able to give access</i> " (X)          | Same, or the app is not set to a confidential client type                              |
| " <i>X isn't configured for native publishing yet</i> " | That platform's keys are empty in <code>.env</code>                                    |
| " <i>X publishes through your Y connection</i> "        | A derived platform — connect Y instead                                                 |
| " <i>A brand must be selected</i> "                     | No brand is open. Open one from Brands first                                           |
| Connect button does nothing / errors instantly          | <code>OAUTH_STATE_SECRET</code> is unset, so the tool is not registered                |
| Everything says <code>via aggregator:stub</code>        | No native keys configured; publishing is going to the stub and nothing is being posted |

An empty key is never an error. `envSet` treats present-but-empty as unset, so an unconfigured platform leaves its tool cleanly unregistered and the button simply is not offered — rather than registering it and failing at call time, which reads to a user as "the product is broken" instead of "this is not set up".